

Notation de De Bruijn et récursion dans le compilateur Faust

Note de synthèse

Résumé. Cette note présente l'utilisation de la notation de De Bruijn dans le compilateur Faust pour représenter les formes récursives produites par l'opérateur $\sim$. Après un rappel de la notation classique en lambda-calcul (section 1), nous décrivons l'adaptation originale qu'en fait Faust : un système de lieu de groupe (binder) avec projections par slot, adapté aux boucles de rétroaction multi-sorties (section 2). Nous détaillons ensuite l'algorithme de conversion des boîtes récursives vers la forme de De Bruijn pendant la phase de propagation, en traitant les cas imbriqués et mutuellement récursifs (section 3). Les formes récursives dégénérées et leur simplification sont abordées en section 4. Enfin, la section 5 décrit la conversion vers la forme symbolique, son algorithme et son rôle pour les phases ultérieures du compilateur.

1. La notation de De Bruijn en lambda-calcul

1.1 Le problème de la liaison de variables

En lambda-calcul classique, les variables liées sont nommées de manière arbitraire. Les termes $\lambda x. x$ et $\lambda y. y$ désignent la même fonction (ils sont *alpha-équivalents*), mais leur représentation syntaxique diffère. Cette ambiguïté nominale pose deux problèmes pratiques pour le traitement automatisé :

- **L'alpha-équivalence** ne peut pas être vérifiée par simple comparaison structurelle : il faut raisonner modulo renommage.
- **La substitution capture-avoiding** est fragile : substituer naïvement un terme contenant des variables libres sous un lieu peut accidentellement *capturer* ces variables, modifiant le sens du terme.

1.2 La solution de De Bruijn (1972)

Dans son article fondateur [de Bruijn 1972], Nicolaas Govert de Bruijn propose de remplacer les noms de variables liées par des entiers naturels. Chaque entier (un *indice de De Bruijn*) encode la distance — mesurée en nombre de lieux traversés — entre l'occurrence de la variable et le lieu qui la lie. Les noms disparaissent entièrement :

$$\begin{aligned}\lambda x. x &\rightarrow \lambda. 1 \\ \lambda x. \lambda y. x &\rightarrow \lambda. \lambda. 2 \\ \lambda x. \lambda y. y &\rightarrow \lambda. \lambda. 1 \\ \lambda x. \lambda y. \lambda z. x \ z \ (y \ z) &\rightarrow \lambda. \lambda. \lambda. 3 \ 1 \ (2 \ 1)\end{aligned}$$

Le principe sémantique est direct :

- 1 désigne le lieu le plus proche ;
- 2 désigne le lieu immédiatement extérieur ;
- et ainsi de suite.

1.3 Propriétés fondamentales

Cette représentation possède deux propriétés clés :

1. **L'alpha-équivalence devient l'égalité structurelle.** Deux termes sont alpha-équivalents si et seulement si leurs représentations de De Bruijn sont identiques. Aucun renommage n'est nécessaire.
2. **La substitution et le lifting deviennent des opérations purement structurelles.** Quand on substitue un terme sous des lieurs supplémentaires, les variables libres du terme substitué doivent être incrémentées pour tenir compte des nouveaux lieurs dans la portée. C'est l'opération de *shift* (ou *lift*), qui est purement mécanique en notation de De Bruijn.

1.4 Indices vs. niveaux

Il existe une formulation duale :

- Les **indices** comptent de l'occurrence de la variable *vers le haut* jusqu'à son lieu (adressage relatif).
- Les **niveaux** comptent de la portée la plus externe *vers le bas* jusqu'au lieu (adressage absolu).

Le compilateur Faust utilise la convention par indices (comptage depuis la référence vers le lieu englobant le plus proche).

1.5 Au-delà du lambda-calcul unaire

La littérature récente montre que la notation de De Bruijn ne se limite pas au lambda-calcul unaire. Keuchel et Jeuring [2012] montrent explicitement que des représentations de De Bruijn bien typées peuvent décrire des lieurs multiples, des portées séquentielles et des portées récursives. La notation a également été étendue aux types récursifs (μ -types), à la réécriture d'ordre supérieur [Bonelli, Kesner, Rios 2000], et aux calculs de processus [Perera, Cheney 2017].

L'idée générale d'utiliser De Bruijn au-delà du lambda-calcul pur n'est donc pas spécifique à Faust. Ce qui l'est, c'est le *type de structure* que Faust choisit d'encoder de cette manière.

2. L'adaptation originale dans le compilateur Faust

2.1 Contexte : l'opérateur $\sim$ et la récursion par rétroaction

En Faust, les programmes sources ne contiennent pas de nœuds DEBRUIJNREC ou DEBRUIJNREF. La récursion est écrite au niveau du langage des boîtes, via l'opérateur $\sim$ (tilde) ou, dans l'API boîte, via `boxRec`. La documentation officielle de Faust met en avant deux faits importants :

- la phase sémantique traduit le programme en signaux par *propagation symbolique* ;
- la récursion insère automatiquement un retard d'un échantillon pour garantir la causalité.

Par exemple :

```
process = + ~ *(0.5);
```

décrit une boucle de rétroaction à un échantillon de retard : la sortie est renvoyée, multipliée par 0.5, et ajoutée à l'entrée.

2.2 Les deux nœuds de De Bruijn dans Faust

La représentation interne utilise deux formes principales :

Nœud	Rôle
DEBRUIJNREC (body)	Lieu d'un groupe récursif. Analogue à λ ou μ .
DEBRUIJNREF (level)	Référence à un lieu englobant. Niveau 1 = le plus interne.

2.3 La spécificité de Faust : le lieu de groupe avec projections

L'adaptation de Faust diffère du lambda-calcul classique sur un point décisif : **le lieu ne lie pas une seule variable, mais un groupe de sorties.**

Une référence récursive seule (DEBRUIJNREF (1)) n'est pas directement utilisable. Elle est presque toujours combinée avec une projection de slot :

```
proj (i, DEBRUIJNREF (1))
```

Et le motif central de rétroaction causale est :

```
delay1(proj (i, DEBRUIJNREF (1)))
```

En d'autres termes, Faust n'utilise pas les indices de De Bruijn pour nommer des variables de lambda-terme, mais pour **identifier le groupe récursif courant à l'intérieur d'un graphe de signaux multi-sorties**. La sélection du slot est déléguée à `proj (i, ...)`.

C'est, à notre connaissance, la spécificité la plus intéressante de cette représentation intermédiaire. Aucune publication de Faust ne documente explicitement cette connexion avec la notation de De Bruijn, bien que le compilateur C++ de référence utilise le même encodage (rec/ref avec niveaux de De Bruijn).

2.4 Pourquoi ne pas utiliser directement des variables nommées ?

Trois raisons pratiques justifient le choix de De Bruijn pendant la phase de propagation :

1. **Partage structurel.** L'arène de termes (TreeArena) interne les nœuds par identité structurelle. Les nœuds de De Bruijn produisent des formes déterministes indépendantes du contexte de nommage, maximisant le partage.
2. **Correction des portées par construction.** Des opérateurs $\sim$ imbriqués produisent des DEBRUIJNREC imbriqués ; les références internes pointent automatiquement vers la bonne portée via leur numéro de niveau — aucune passe d'alpha-renommage n'est nécessaire.
3. **Technique standard.** Le compilateur C++ de Faust utilise le même encodage, ce qui assure la parité structurelle avec le port Rust.

3. Conversion des boîtes récursives vers la notation de De Bruijn

3.1 Le cadre : la composition récursive $A \sim B$

Au niveau de l'algèbre de boîtes, $A \sim B$ construit une composition récursive. Si :

- $A : Li \rightarrow Lo$ (le corps principal)
- $B : Ri \rightarrow Ro$ (le chemin de rétroaction)

alors la composition est bien formée quand $Ri \leq Lo$ et $Ro \leq Li$, et le résultat a pour arité $(Li - Ro) \rightarrow Lo$.

Intuition : - B lit certaines sorties de A ; - B produit des signaux de rétroaction qui alimentent certaines entrées de A ; - la rétroaction est toujours retardée d'un échantillon, donc le cycle est causal.

3.2 L'algorithme de propagation pas à pas

Quand le propagateur rencontre un nœud `FlatNodeKind::Rec(left, right)`, il exécute les étapes suivantes :

Étape 1 — Vérification des arités.

```
left  : Li → Lo
right : Ri → Ro
Exiger : Ri ≤ Lo ET Ro ≤ Li
```

Étape 2 — Création des placeholders de rétroaction. Pour chacun des Ri canaux de rétroaction, créer un signal placeholder :

```
l0[i] = delay1(proj(i, DEBRUIJNREF(1)))    pour i = 0..Ri-1
```

Cela signifie : « la i -ème entrée de rétroaction est l'échantillon précédent (`delay1`) de la i -ème projection (`proj`) du groupe récursif en cours de définition (`DEBRUIJNREF(1)`) ».

Étape 3 — Propagation du chemin de rétroaction.

```
l1 = propagate(right, l0)
```

Étape 4 — Construction du vecteur d'entrées complet.

```
rec_inputs = l1 ++ lift(external_inputs)
```

Le corps `left` reçoit d'abord les Ro signaux de rétroaction, puis les $Li - Ro$ entrées externes, **liftées** d'un niveau de De Bruijn.

Étape 5 — Lifting de l'environnement de slots.

```
slot_env' = { k → liftn(v, 1) | (k, v) ∈ slot_env }
```

Toute valeur de l'environnement de slots contenant des nœuds `DEBRUIJNREF` doit être liftée pour éviter la capture par le nouveau lieu interne.

Étape 6 — Propagation du corps.

```
l2 = propagate(left, rec_inputs)    // avec slot_env'
```

Étape 7 — Enveloppement et projection.

```

group = DEBRUIJNREC(list(l2[0], l2[1], ..., l2[Lo-1]))

output[i] =
  si aperture(l2[i]) > 0 : proj(i, group)    // vraiment récursif
  sinon                  : l2[i]            // forme fermée, émis directement

```

3.3 Exemple simple : $+ \sim *(0.5)$

```
process = + ~ *(0.5);
```

Le compilateur construit d'abord le seed de rétroaction :

```
delay1(proj(0, DEBRUIJNREF(1)))
```

Le chemin de rétroaction $*(0.5)$ le transforme, et le corps $+$ construit :

```

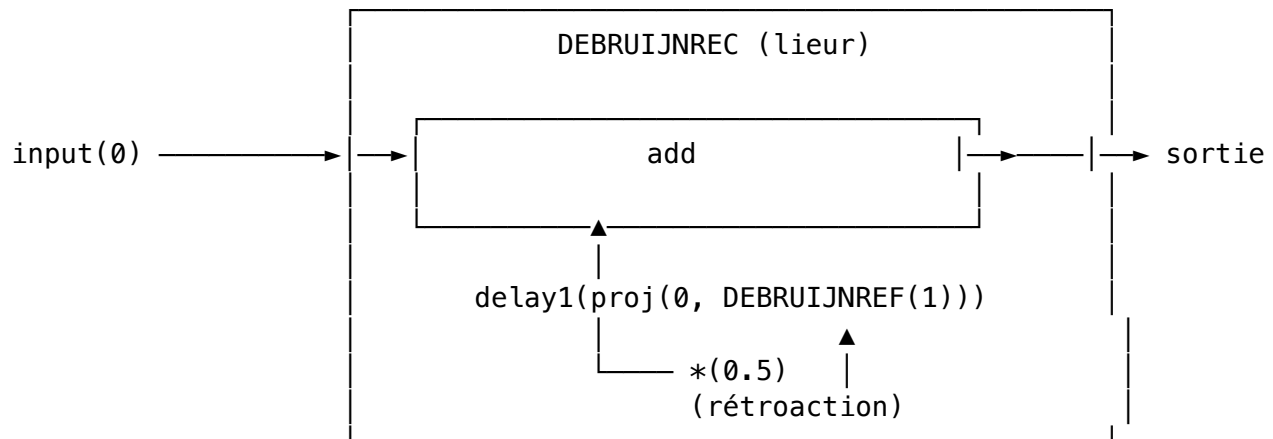
body0 = add(
  delay1(proj(0, DEBRUIJNREF(1))) * 0.5,
  input(0)
)

```

```
group = DEBRUIJNREC([body0])
```

```
out0 = proj(0, group)
```

Diagramme :



3.4 Récursions imbriquées

Considérons une forme où une récursion est elle-même placée dans une autre :

```

inner = + ~ *(0.5);
process = inner ~ *(0.25);

```

La récursion définie par `inner` est elle-même placée sous un nouveau lieu récursif. La conséquence est le comportement classique de De Bruijn : toute référence libre provenant de la portée externe doit être **liftée d'un niveau** pour ne pas être capturée par le lieu interne.

Schématiquement, on obtient une structure de la forme :

```

DEBRUIJNREC(                                ← groupe externe
  ...
  DEBRUIJNREC(                                ← groupe interne
    pair(
      DEBRUIJNREF(1),                        ← pointe vers le groupe interne
      DEBRUIJNREF(2)                        ← pointe vers le groupe externe
    )
  )
  ...
)

```

L'algorithme de lifting (liftn) :

```

liftn(node, threshold):
  si node = DEBRUIJNREF(level):
    retourner DEBRUIJNREF(level)      si level < threshold
    retourner DEBRUIJNREF(level + 1)  sinon

  si node = DEBRUIJNREC(body):
    retourner DEBRUIJNREC(liftn(body, threshold + 1))

  sinon:
    reconstruire le nœud en appliquant liftn à chaque enfant

```

Le `threshold + 1` est le point clé. Descendre sous un lieu récursif interne signifie qu'un niveau supplémentaire devient localement lié. Le critère de « liberté » doit donc se décaler.

Exemple concret. Supposons qu'une valeur provenant d'une récursion externe contienne déjà :

```
delay1(proj(0, DEBRUIJNREF(1)))
```

et que cette valeur soit réutilisée dans un nouveau Rec. Sans lifting, `DEBRUIJNREF(1)` serait maintenant interprété comme « le nouveau groupe interne », alors qu'il signifiait « le groupe externe existant ».

Après `liftn(..., 1)` :

```
delay1(proj(0, DEBRUIJNREF(2)))
```

Le sens lexical est préservé : - `DEBRUIJNREF(1)` désigne désormais le nouveau groupe interne ;
- `DEBRUIJNREF(2)` continue de désigner l'ancien groupe externe.

Pourquoi Faust doit lifter en deux endroits. Dans `propagate`, cette opération est appliquée à deux familles d'objets :

1. Les **entrées** injectées dans le corps `left` du Rec ;
2. Les **valeurs du `slot_env`** (environnement de slots).

Le second point est facile à manquer mais essentiel. Une valeur produite par une abstraction de boîte ou une définition locale peut contenir des nœuds `DEBRUIJNREF` provenant d'une boucle externe. Si elle est réinjectée sans lifting dans une boucle interne, elle sera silencieusement capturée par le mauvais lieu.

3.5 Récursion multi-sortie et récursion mutuellement récursive

En Faust, la **récursion mutuelle** est un cas particulier de la **récursion multi-sortie**, pas un synonyme. Les deux utilisent le même mécanisme : un seul lieu DEBRUIJNREC enveloppant un vecteur de corps.

```
import("stdfaust.lib");
feedback = si.bus(2) ~ ((*0.5), *(0.25));
process = feedback;
```

Récursion multi-sortie (chaque canal se nourrit lui-même) Cet exemple représente un groupe récursif à deux corps :

```
DEBRUIJNREC([
  body0 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5,
  body1 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25
])
```

Chaque canal dépend de son propre slot : body0 projette le slot 0, body1 projette le slot 1. C'est une récursion multi-sortie, mais pas une récursion mutuelle au sens strict.

Récursion véritablement mutuelle (les signaux se croisent) Pour obtenir une vraie récursion mutuelle, il faut croiser les signaux à l'intérieur de la boucle récursive. L'opérateur `ro.cross(2)` permute deux signaux :

```
import("stdfaust.lib");
process = si.bus(2) ~ ((*0.5), *(0.25)) : ro.cross(2));
```

Le croisement fait que chaque sortie dépend de l'*autre* sortie :

```
DEBRUIJNREC([
  body0 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25,
  body1 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5
])
```

Sémantiquement, les deux signaux sont couplés :

$$\begin{aligned} \text{output0}[n] &= 0.25 \times \text{output1}[n-1] \\ \text{output1}[n] &= 0.5 \times \text{output0}[n-1] \end{aligned}$$

Le point clé Le lieu est partagé par le vecteur de sorties entier, pas un lieu par sortie. Les sorties mutuellement récursives ne sont pas représentées par un mécanisme différent — elles sont un cas particulier de groupe multi-sortie avec des projections croisées. Du point de vue du compilateur, la seule différence est *quelles projections apparaissent dans chaque corps*.

4. Formes récursives dégénérées

4.1 Qu'est-ce qu'une forme dégénérée ?

Une forme récursive est dite *dégénérée* quand le groupe récursif matériel existe toujours, mais une ou plusieurs de ses sorties **ne dépendent plus réellement de la rétroaction**. Cela se produit quand certaines branches du chemin de rétroaction :

- rejettent explicitement leur entrée récursive ;
- ou deviennent fermées après propagation et simplification.

4.2 Exemple représentatif

Le cas classique est un bus multi-canal où la plupart des canaux ignorent leur rétroaction :

```
import("stdfaust.lib");

N = 8;
gain = hslider("gain", 0.5, 0.0, 0.99, 0.01);

process = si.bus(N) ~ (!, !, !, !, !, !, !, *(gain));
```

Ici, les canaux 0 à 6 rejettent leur entrée récursive via ! ; seul le canal 7 reste véritablement récursif. Le déclencheur réel de ce problème dans le monde réel a été `re.zita_rev1_stereo(...)` (fichier `Birds.dsp`), un reverb algorithmique à 8 lignes de retard dont la matrice de rétroaction produisait exactement cette forme après évaluation et propagation.

4.3 Détection par l'aperture

L'outil conceptuel pour détecter les branches dégénérées est l'**aperture**, définie comme le niveau maximum de référence de De Bruijn libre dans un sous-arbre :

```
aperture(DEBRUIJNREF(k))      = k
aperture(DEBRUIJNREC(body))   = aperture(body) - 1
aperture(autre nœud)          = max(aperture(enfants))
```

Interprétation : - `aperture > 0` : la branche est encore ouverte sur le groupe récursif ; - `aperture ≤ 0` : la branche est fermée à cette frontière.

Dans `propagate`, une sortie fermée n'est plus émise comme `proj(i, group)` mais directement comme expression brute.

4.4 Le problème des indices de projection décalés

La détection par `aperture` ne suffit pas à éliminer toute dégénérescence structurelle. Dans le pipeline C++ classique, une passe plus agressive, `inlineDegenerateRecursions()`, peut compacter un groupe et ne conserver que les corps véritablement récursifs. Cela crée un problème subtil : l'indice logique de projection peut rester l'original, tandis que l'arité physique du groupe a diminué :

```
avant compaction : proj(7, SYMREC([b0, ..., b7]))
après compaction  : proj(7, SYMREC([b7]))
```


Le groupe n'a plus qu'un corps physique, mais la projection dit toujours 7. Pour un backend qui modélise les slots récursifs comme un `Vec`, c'est un indice hors limites.

4.5 La simplification dans le port Rust

Dans le port Rust actuel, la simplification adoptée est plus étroite : dans `signal_prepare`, toute projection ciblant un groupe symbolique unaire est **canonicalisée** en `proj(0, group)` :

avant : `SYMREC(W, [body_7])` avec `proj(7, W)`
 après : `SYMREC(W, [body_7])` avec `proj(0, W)`

L'enjeu n'est pas cosmétique. Simplifier les formes dégénérées permet de :

- maintenir des indices de slot denses ;
- stabiliser le typage et la génération FIR ;
- éviter les erreurs « projection index out of bounds » en aval.

5. Conversion de la forme de De Bruijn vers la forme symbolique

5.1 Pourquoi cette conversion ?

La forme de De Bruijn est excellente pendant la propagation :

- aucune génération de noms n'est nécessaire ;
- les portées sont correctes par construction ;
- le partage structurel dans l'arène est maximisé.

Cependant, elle n'est ni très lisible ni très pratique pour les passes ultérieures. Compter mentalement les profondeurs de lieux à l'intérieur d'un DAG de signaux partagés est tolérable pour `propagate`, beaucoup moins pour le typage, la préparation FIR, la génération de code récursif, et les diagnostics.

La forme symbolique remplace donc :

Forme de De Bruijn	Forme symbolique
<code>DEBRUIJNREC(body)</code>	<code>SYMREC(var, body)</code>
<code>DEBRUIJNREF(level)</code>	<code>SYMREF(var)</code>

5.2 L'algorithme de conversion

L'algorithme `de_bruijn_to_sym(...)`, partagé entre le compilateur C++ historique et le port Rust, est conceptuellement :

```

convert(node):
  si node = DEBRUIJNREC(body):
    var = fresh("W") // ex: W0, W1, ...
    body1 = substitute(body, level=1, replacement=SYMREF(var))
    body2 = convert(body1)
    retourner SYMREC(var, body2)

```

```

si node = DEBRUIJNREF(level):
    erreur : l'arbre converti est ouvert

```

```

sinon:
    reconstruire le nœud récursivement

```

Le point subtil est la **substitution** :

```

substitute(node, level, repl):
    si aperture(node) < level:
        retourner node                // pas de référence libre à ce niveau
    si node = DEBRUIJNREF(level):
        retourner repl                // remplacement direct
    si node = DEBRUIJNREC(body):
        retourner DEBRUIJNREC(substitute(body, level + 1, repl))
    sinon:
        reconstruire en appliquant substitute aux enfants

```

Quand on descend sous un DEBRUIJNREC imbriqué, le niveau cherché devient `level + 1`, exactement comme dans `liftn`. C'est ce qui rend possible la conversion correcte des arbres imbriqués.

5.3 Exemple avec récursion imbriquée

L'entrée :

```

DEBRUIJNREC(
    DEBRUIJNREC(
        pair(DEBRUIJNREF(1), DEBRUIJNREF(2))
    )
)

```

La conversion produit :

```

SYMREC(W0,
    SYMREC(W1,
        pair(SYMREF(W1), SYMREF(W0))
    )
)

```

La logique lexicale attendue est retrouvée : - DEBRUIJNREF(1) dans le groupe interne → SYMREF(W1) (lieur le plus proche) ; - DEBRUIJNREF(2) dans le même groupe → SYMREF(W0) (lieur externe).

5.4 Propriétés de la conversion

La conversion est un **changement de représentation**, pas une normalisation structurelle :

- l'imbrication des lieurs reste identique ;
- la liste de corps récursifs reste identique ;
- les indices de projection (`proj(i, ...)`) restent identiques ;

- seul le porteur de la récursion change, passant de références positionnelles à des références nommées.

5.5 Utilisation par les phases ultérieures

Dans le port Rust, `prepare_signals_for_fir(...)` clone la forêt de sortie entière, applique `de_bruijn_to_sym(...)` à la liste complète, puis effectue dans l'ordre :

1. La conversion de De Bruijn vers symbolique ;
2. La canonicalisation des projections unaires dégénérées ;
3. L'inférence de type réduite (Int/Real/Sound) ;
4. La promotion des casts de signaux ;
5. La préparation FIR.

Le lowerer FIR attend ensuite des groupes de la forme :

`SYMREC(var, body_list)`

`SYMREF(var)`

et plus aucun nœud `DEBRUIJNREC` / `DEBRUIJNREF`. Ce choix apporte trois bénéfices pratiques :

- il rend l'identité du groupe récursif explicite ;
- il permet à `signal_fir` de décoder directement la liste de corps d'un groupe symbolique ;
- il établit une frontière de pipeline propre : après la préparation, le backend n'a plus besoin de raisonner en termes de profondeurs lexicales.

En résumé, la conversion n'est pas simplement cosmétique. C'est un changement de représentation qui sépare la **logique de portée** (utile pendant la propagation) de la **logique de consommation backend** (utile pendant le lowering).

6. Conclusion

Dans le compilateur Faust, la notation de De Bruijn joue un rôle plus concret que dans de nombreuses présentations purement théoriques : elle sert de forme de travail pour construire des groupes de rétroaction corrects par construction, même en présence d'imbrication, de groupes multi-sorties, et de partage structurel.

Le point le plus important n'est pas simplement « Faust utilise De Bruijn », ce qui serait trop général, mais plutôt :

- Faust traite la récursion comme un **lieur de groupe**, pas comme une variable unique ;
- les références récursives sont adressées d'abord par **profondeur**, puis par **projection de slot** ;
- les formes dégénérées imposent une discipline de **canonicalisation** ;
- la conversion finale vers `SYMREC` / `SYMREF` **isole** les passes aval de la complexité des portées.

De ce point de vue, la forme de De Bruijn dans Faust est à la fois classique dans son principe et hautement spécialisée dans son usage compilateur.

Références

Sources externes

- N. G. de Bruijn, « Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem », *Indagationes Mathematicae*, vol. 34, pp. 381-392, 1972. <https://research.tue.nl/en/publications/lambda-calculus-notation-with-nameless-dummies-a-tool-for-automat-2/>
- S. Keuchel, J. T. Jeuring, « Generic Conversions of Abstract Syntax Representations », WGP 2012. <https://ics-archive.science.uu.nl/research/techreps/repo/CS-2012/2012-009.pdf>
- E. Bonelli, D. Kesner, A. Rios, « A de Bruijn notation for higher-order rewriting », RTA 2000. https://doi.org/10.1007/10721975_5
- Y. Orlarey, D. Fober, S. Letz, « Syntactical and Semantical Aspects of Faust », *Soft Computing*, vol. 8, 2004. <https://link.springer.com/article/10.1007/s00500-004-0388-1>
- Y. Orlarey, S. Letz, D. Fober, R. Michon, « A New Intermediate Representation for Compiling and Optimizing Faust Code », International Faust Conference, 2020. <https://hal.science/hal-03124677>
- Documentation Faust, « Using the box API ». <https://faustdoc.grame.fr/tutorials/box-api/>

Sources internes au dépôt

- `debruijn-recursion-faust-note-en.md` — note détaillée sur De Bruijn et la récursion dans Faust.
- `recursion-debruijn-lowering-en.md` — document de design interne sur le lowering.
- `flatnode-rec-to-signals-en.md` — description opérationnelle de la conversion `FlatNodeKind::Rec` vers signaux.
- `crates/propagate/src/lib.rs` — implémentation de la propagation.
- `crates/tlib/src/recursion.rs` — conversion de De Bruijn vers symbolique, lifting, aperture.
- `crates/transform/src/signal_prepare.rs` — canonicalisation des récursions dégénérées unaires.
- `tests/corpus/rep_71_degenerate_unary_recursion.dsp` — régression pour les formes dégénérées.
- `tests/corpus/rep_79_multi_output_recursion.dsp` — régression pour la récursion multi-sortie.
- `tests/corpus/rep_80_mutual_recursion_crossed.dsp` — régression pour la récursion mutuellement récursive.